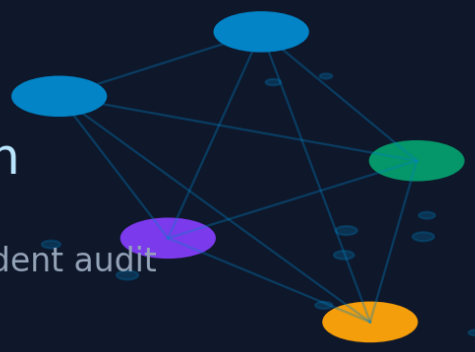
NEXORA

Multi-Business Operating Platform

Multi-entity ERP | Role-based access | Tamper-evident audit

Frontend prototype: React 19 + TypeScript + Vite + Tailwind CSS



Project Documentation & Technical Overview

A detailed guide to the NEXORA multi-entity ERP prototype: purpose, architecture, modules, security model, data model and usage

Contents

1. Executive Summary

2. Technology Stack and Project Layout

3. Architecture

4. The Three Demo Businesses

5. Functional Modules

6. Security and Governance Model

7. Data Model

8. Inter-Company Engine

9. Nexora Studio (Business Onboarding)

10. Demo Personas and Access Passport

11. Running, Building and Verifying

12. Limitations and Production Roadmap

This document was written from a direct review of the source code in the uploaded archive (React components, context, seed data, type definitions and security libraries). Where a figure comes from seed data it is labelled as such.

1. Executive Summary

NEXORA is a browser-based ERP prototype for running several legally separate businesses from a single interface. It combines finance, inventory, sales and procurement, HR and payroll, inter-company accounting, approvals and a tamper-evident audit log, all organised around the idea that every business entity is its own protected tenant. A group-level command center sits above the entities and offers a consolidated view to users who hold group roles.

The project is explicitly a **frontend prototype**. It ships with seeded sample data for three fictional companies (a US manufacturer, a German retailer and a UK advisory firm), five role-based demo personas, and persists its state in the browser's `localStorage`. There is no backend, database or server-enforced authentication; the security workflows demonstrate how such controls would behave rather than providing them.

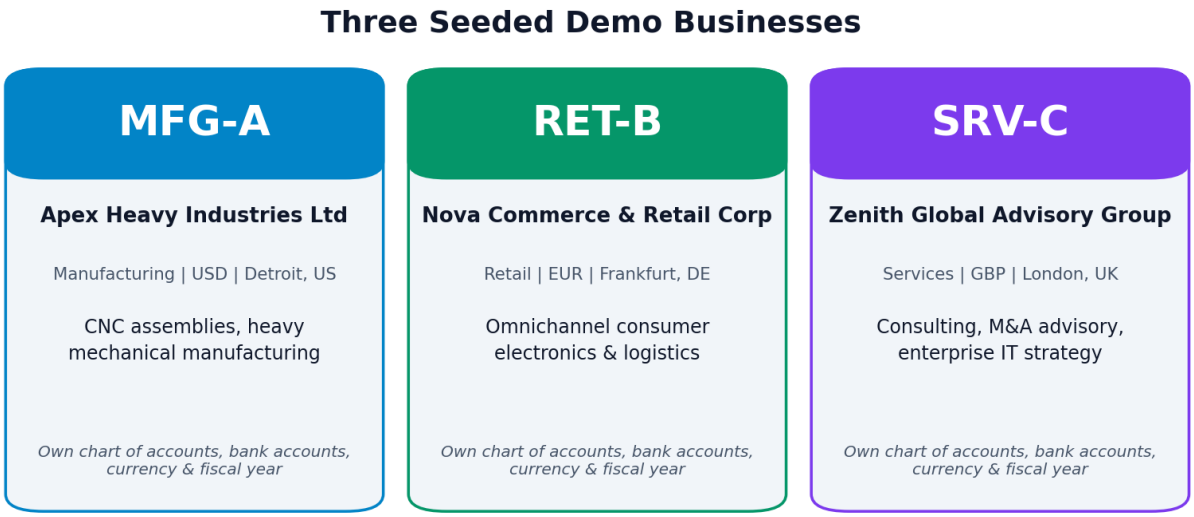


Figure 1 - The three seeded demo businesses, each with its own currency, chart of accounts and bank accounts.

Headline capabilities

- **Multi-entity operation** with per-entity currency (USD, EUR, GBP), tax ID, fiscal year start and bank accounts.
- **Group Command Center** for consolidated reporting across entities, restricted to group-level roles.
- **Core ERP modules**: accounting (chart of accounts, journals, AR invoices), inventory, buying and selling, HR and payroll.
- **Inter-company engine** that mirrors a trade between two entities and tracks reconciliation and elimination.
- **Access Passport**: per-entity access grants that can be locked, pending, approved or time-boxed, plus an emergency break-glass override.
- **Approval engine** with a queue, SLA deadlines and segregation of duties on purchase orders (the creator cannot approve their own PO).
- **Audit Vault**: SHA-256 hash-chained log with a built-in verifier and a tamper simulation.
- **Nexora Studio** to launch a new business from an industry template.
- **Usability extras**: light/dark themes, animated loading screen, quick-start guide, a built-in help manual and a custom right-click menu.

2. Technology Stack and Project Layout

The application is a single-page React app built with Vite. Dependencies are intentionally small: the UI is composed with React and Tailwind CSS, icons come from lucide-react and animation from the motion library. All business logic is plain TypeScript running in the browser.

Area	Technology (from package.json)	Role
UI framework	React 19 and React DOM	Component model and state via Context
Language	TypeScript	Typed domain models and components
Build tool	Vite with @vitejs/plugin-react	Dev server (port 3000) and production build
Styling	Tailwind CSS 4 via @tailwindcss/vite	Utility-first styling, dark mode
Icons / motion	lucide-react, motion	Navigation icons and animations
Crypto	Native Web Crypto API (SubtleCrypto)	SHA-256 for the audit chain
Runtime	Node.js 20.19+ (or 22.12+), npm	Required for development and build

Source tree

`src/` is organised by responsibility:

Path	Contents
src/App.tsx	Root layout; switches the main workspace based on the active tab and hosts modals and the toast.
src/context/AppContext.tsx	Central provider: all state, persistence to localStorage and business workflows (about 900 lines).
src/components/	One folder per feature: command-center, finance, inventory, sales-procurement, hr, intercompany, approvals, audit, security, studio, passport, help, layout, common.
src/data/seedData.ts	Seed entities, users, accounts, journals, inventory, customers, vendors, purchase orders, invoices, employees, leave requests, templates and the genesis audit chain.
src/lib/security.ts	Role-permission map, 3-dimensional authorization, segregation of duties and field masking.
src/lib/crypto.ts	SHA-256 hashing, audit block hashing and full chain verification.
src/types/	Domain types: auth, entity, finance, inventory, sales_procurement, hr, intercompany, approvals, audit, templates.

3. Architecture

NEXORA follows a simple client-side layered design. `main.tsx` mounts App, which wraps the interface in an `AppProvider`. The provider owns every piece of state and exposes it through the `useApp()` hook. Feature modules read state and call workflow functions; they do not talk to each other directly.

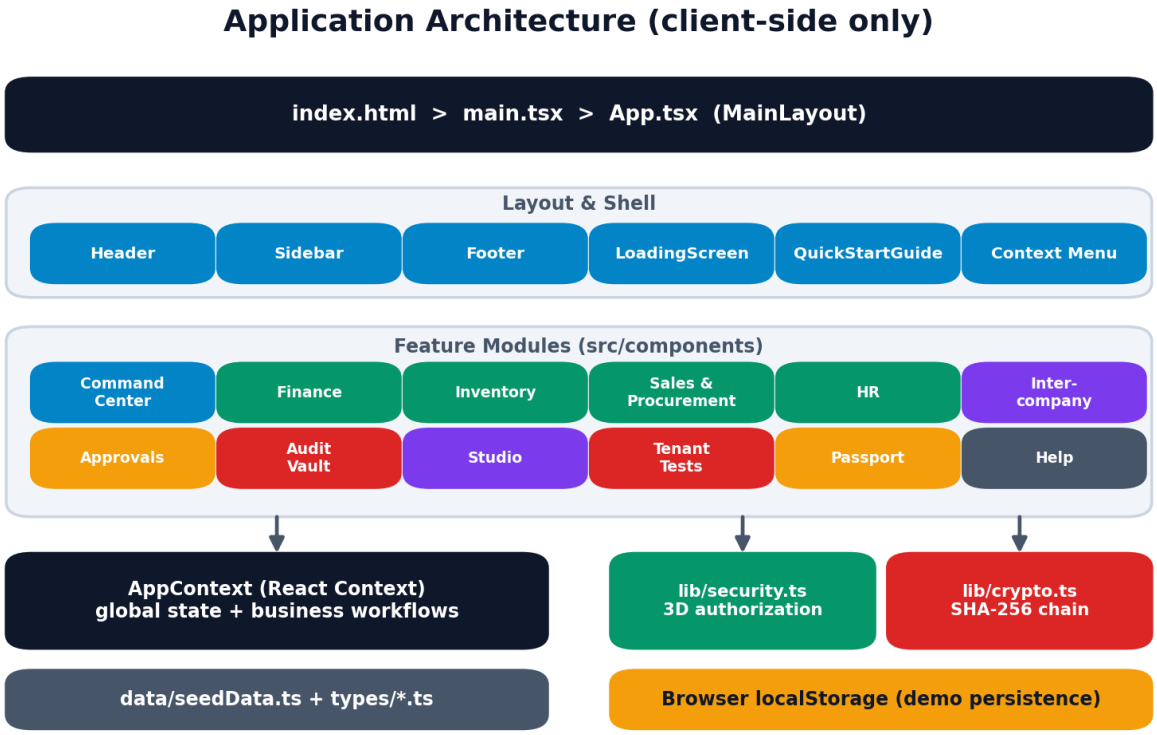


Figure 2 - Layers of the application: shell, feature modules, shared context and libraries, data and persistence.

State and persistence

The provider keeps entities, users, access requests, audit logs, accounts, journals, invoices, inventory, purchase orders, leave requests, inter-company transactions and approval items in React state. Each of these is initialised from `localStorage` keys prefixed with `nexora_` (for example `nexora_entities`, `nexora_audit_logs`, `nexora_pos`) and falls back to seed data on first run. A `resetToSeedData` utility restores the original demo state. Warehouses, customers, vendors, employees and templates are held as static seeded collections.

Navigation

The sidebar groups twelve destinations under four headings. The active tab is a typed union (`ActiveTab`) so the router in `App.tsx` is a simple conditional render. The sidebar shows a live badge with the number of pending approvals.

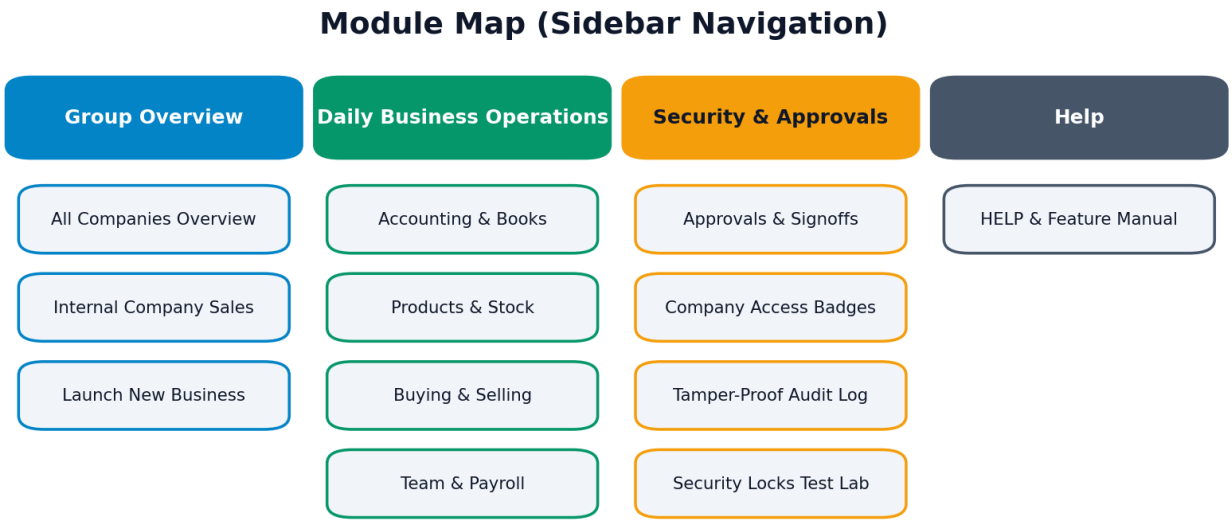


Figure 3 - Sidebar navigation grouped by purpose.

4. The Three Demo Businesses

Each entity has its own base currency, tax ID, headquarters, bank accounts and chart of accounts. The differing currencies exist to exercise the inter-company and consolidation logic.

Code	Name	Type	Currency	HQ	Focus
MFG-A	Apex Heavy Industries Ltd	Manufacturing	USD	Detroit, Michigan	Precision industrial equipment, CNC assemblies
RET-B	Nova Commerce & Retail Corp	Retail	EUR	Frankfurt, Germany	Omnichannel consumer electronics, logistics
SRV-C	Zenith Global Advisory Group	Services	GBP	London, UK	Strategy consulting, M&A advisory, IT strategy

Seeded chart of accounts

The seed data lists accounts by category (asset, liability, equity, revenue, expense). Balances below are the revenue and expense accounts as they appear in the seed file, in each entity's local currency. They illustrate the scale of the demo data and are not audited figures.

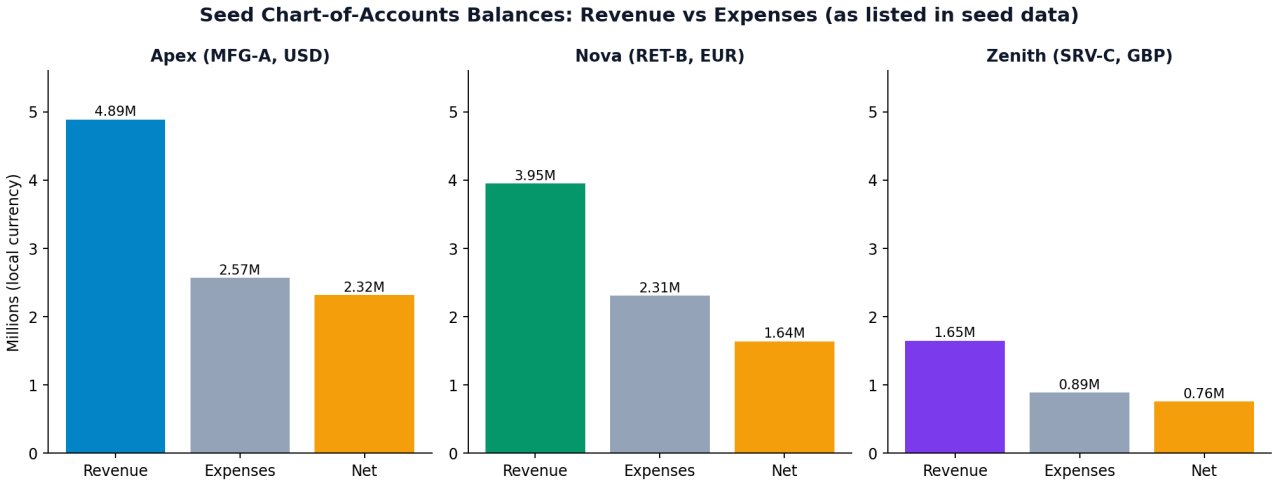


Figure 4 - Revenue and expense balances from the seed chart of accounts (local currency, millions). Net is revenue minus the listed expense accounts.

Other seeded records include four warehouses (two in Michigan for Apex, two in Germany for Nova), four inventory items (such as a titanium 5-axis CNC milling spindle, a 120 kN hydraulic actuator, an IoT gateway router and an industrial touch display), three customers, two vendors, purchase orders, invoices, employees and leave requests.

5. Functional Modules

Each module is a React component that respects the active entity selector in the header (a single entity or the whole group) and the active persona's permissions.

Module	What it does
Group Command Center	The landing view. Shows entity-level KPI cards (revenue, expenses, cash, open invoices, pending approvals, stock valuation, headcount) and consolidated views for group roles. Selecting an entity scopes the other modules to it.
Accounting and Books (Finance)	Three tabs: Chart of Accounts, Journal Entries and Invoices. Supports journal entries with balanced debit and credit lines, AR invoices with line items and tax, AP bills, and per-entity fiscal period locking. Posting actions write to the audit log.
Products and Stock (Inventory)	Item master with SKU, unit of measure, valuation method (FIFO or weighted average), cost and selling price, reorder point and quantity, and stock by warehouse. Cost price is masked for roles such as sales executive. Includes stock adjustments and inter-company stock transfers.
Buying and Selling (Sales and Procurement)	Three tabs: Procurement, Sales and Vendors. Purchase orders move through draft, pending CFO or admin approval, approved, goods received and billed, with a three-way-match status (matched, discrepancy, pending GRN, pending bill). Sales orders move from draft to confirmed, fulfilled and invoiced.
Team and Payroll (HR)	Employee directory, leave requests with approval, and payroll runs (gross pay, deductions, net pay, headcount, status). Salary and bank account values are masked unless the viewer's role is permitted.
Internal Company Sales (Inter-Company)	Posts trades between entities, mirrors them as a bill on the counterparty, reports pair balances and reconciliation differences and lists elimination entries. See Section 8.
Approvals and Signoffs	A queue of pending items (purchase orders, payments, journal entries, access requests) with SLA deadlines, required role and approve or reject actions. Purchase order and access-request decisions are routed to their own workflows (PO approval enforces segregation of duties).
Launch New Business (Studio)	Wizard to create a new entity from an industry template. See Section 9.
Company Access Badges (Passport)	Shows the current user's per-entity grants and lets them request access to locked entities with a justification and duration.
Tamper-Proof Audit Log	Hash-chained log viewer with a verify button and a tamper-injection demonstration.
Security Locks Test Lab	Runs six automated isolation and authorization scenarios and reports pass or fail.
HELP and Feature Manual	A large built-in manual (the sidebar advertises 26 guides) explaining each feature in plain language.



Figure 5 - Illustrative layout of the command center (schematic drawing, not a screenshot): header, sidebar, entity cards, trend and approvals panels.

6. Security and Governance Model

Security is the most distinctive part of the project. It is implemented in `src/lib/security.ts` and `src/lib/crypto.ts`, and wired into the workflows in the context provider.

6.1 Three-dimensional authorization

The function `check3DAuthorization(user, entityId, permission, breakGlass)` decides whether an action is allowed. In order:

- **Consolidated reports** (`report.consolidated`) are allowed only for platform owner, group CEO, CFO, auditor or controller.
- **Break-glass**: if an emergency session is active, the check passes with a note that a temporary override is in effect.
- **Dimension 2 - entity boundary**: the user must hold an Access Passport grant for the entity. Locked, pending and expired grants are denied, and a grant with an `expiresAt` date in the past is denied.
- **Dimension 1 - role permission**: the role on that grant must include the requested permission key.

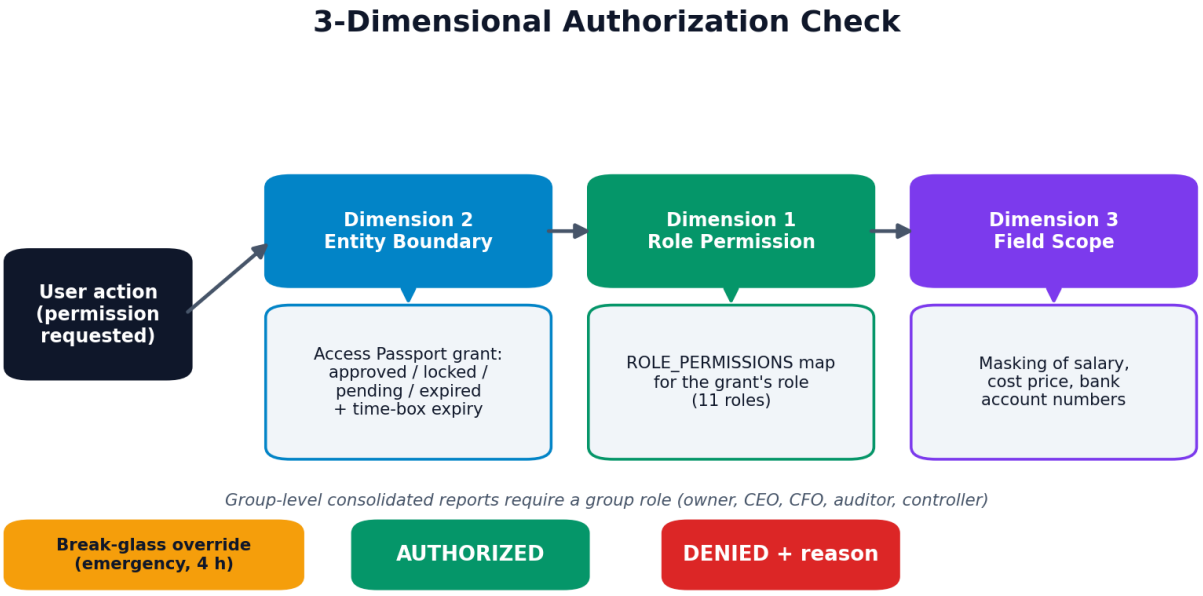


Figure 6 - Authorization flow: entity boundary, role permission and field-level scope, with a break-glass exception.

Dimension 3 - field scope is applied at display time by `maskSensitiveValue`: salary is visible only to HR manager, business admin and group CFO; cost price is hidden from sales executives (or when the grant scope sets `hideCostPrice`); bank account numbers always show only the last four digits.

6.2 Roles and permissions

There are eleven platform roles. The permission map defines 30 permission keys across platform, finance, sales, procurement, inventory, HR, reporting, inter-company, audit and break-glass areas. The chart shows how many keys each role holds; the business admin is the broadest entity-level role, while group roles are mostly read-and-approve oriented.

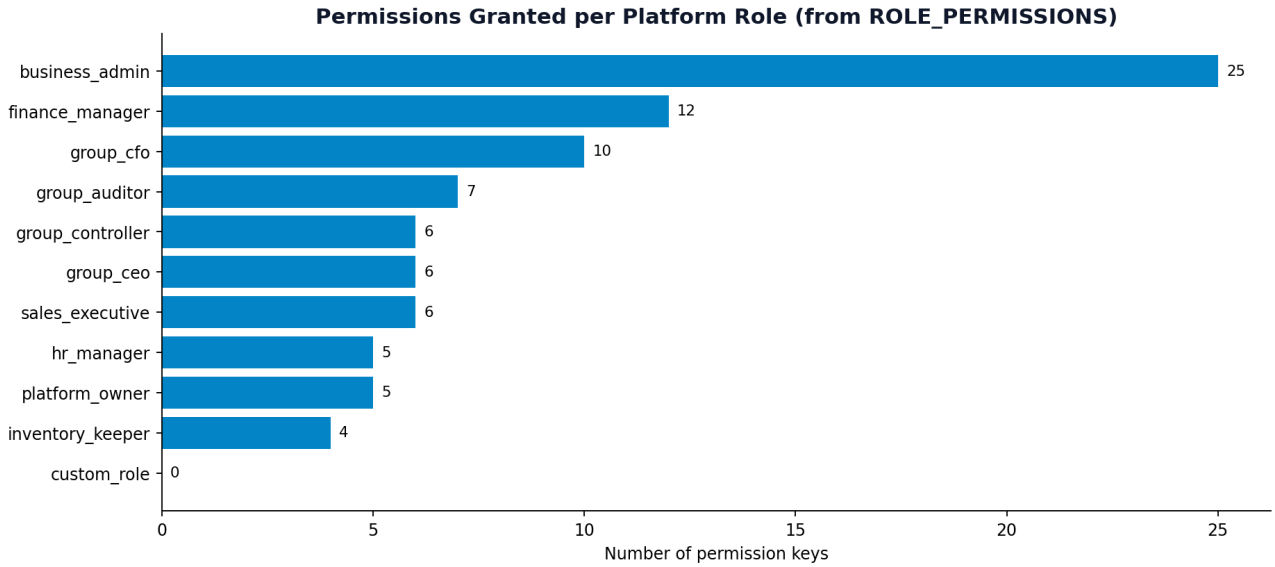


Figure 7 - Number of permissions per role, computed from the ROLE_PERMISSIONS map in the source.

6.3 Segregation of duties

`validateSegregationOfDuties(creatorId, approverId)` returns a violation if both IDs match. In the current code it is enforced in purchase order approval (the approval queue routes PO decisions through that same function), so a PO creator cannot approve their own order. Other queue item types

(payments, journal entries) are resolved without this check in the prototype, which is one gap to close in production. The approval rules model also carries a threshold amount, required role and escalation hours.

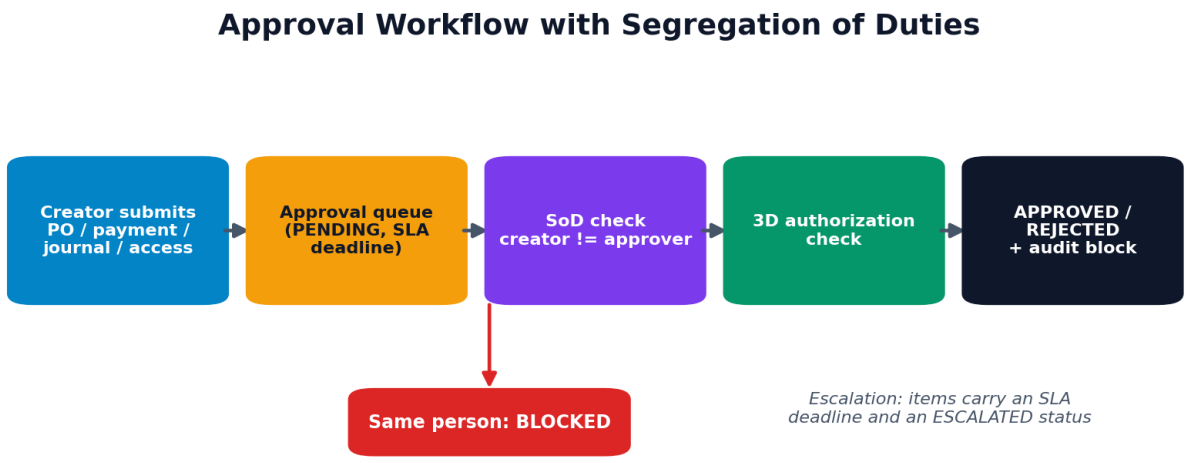


Figure 8 - Intended approval lifecycle: submit, queue, SoD check, authorization check, decision and audit record (SoD is enforced for POs in the current code).

6.4 Access Passport and break-glass

Every user carries an *access passport*: one grant per entity with a role, a status (approved, locked, pending, expired), optional expiry, a justification and an optional scope. Locked users can submit an access request that appears in the approval queue; approving it creates a time-boxed grant based on the requested duration. For emergencies, a break-glass session can be started for an entity with a stated reason; the code sets it to expire after four hours, broadcasts an alert flag and writes a BREAK_GLASS audit block.

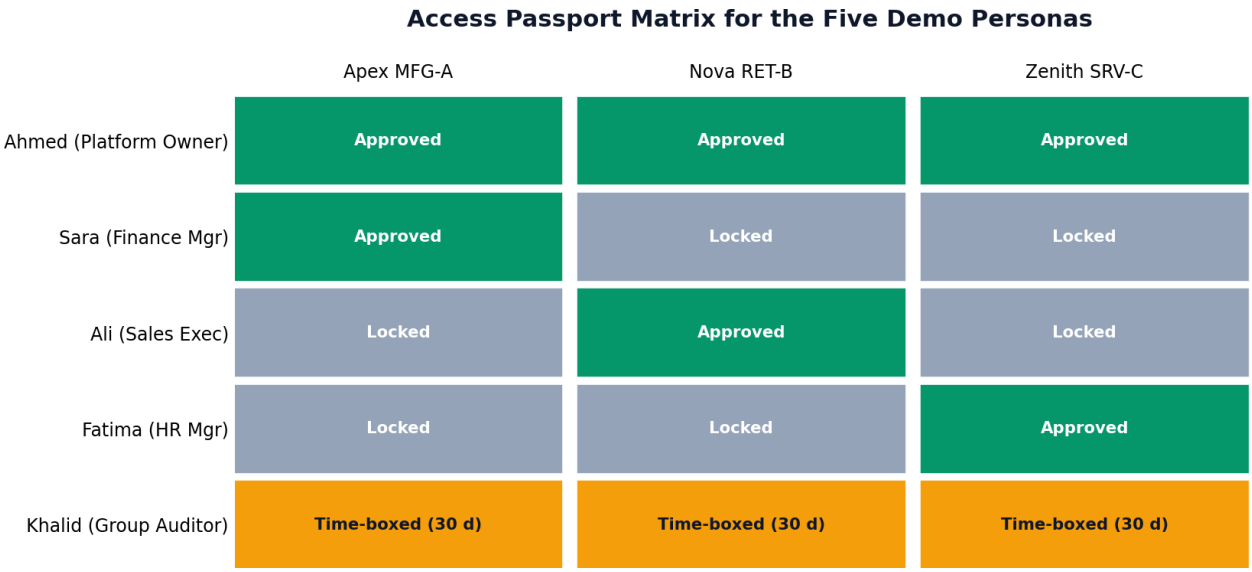


Figure 9 - The five demo personas and their passport status per entity (from seed data).

6.5 Tamper-evident Audit Vault

Every significant action calls `appendAuditLog`, which creates a block containing an index, the previous block's hash, a UTC timestamp, actor, entity, action, module, description and before/after state, and hashes the canonical string with SHA-256. `verifyAuditChain` walks the chain, checks index order and previous-hash links and recomputes every hash. The first block links to 64 zeros. The UI includes a tamper-injection function so a reviewer can modify a block and watch verification fail at the exact index.

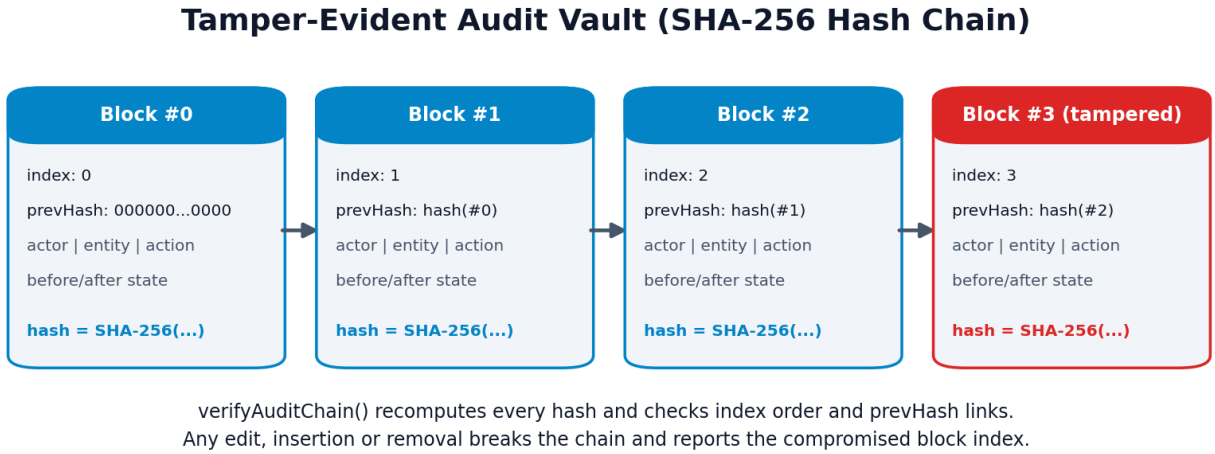


Figure 10 - Hash-chained audit blocks; modifying any block invalidates it and breaks the links that follow.

6.6 Security Locks Test Lab

The test lab turns the security claims into runnable checks, so a reviewer can demonstrate them with one click:

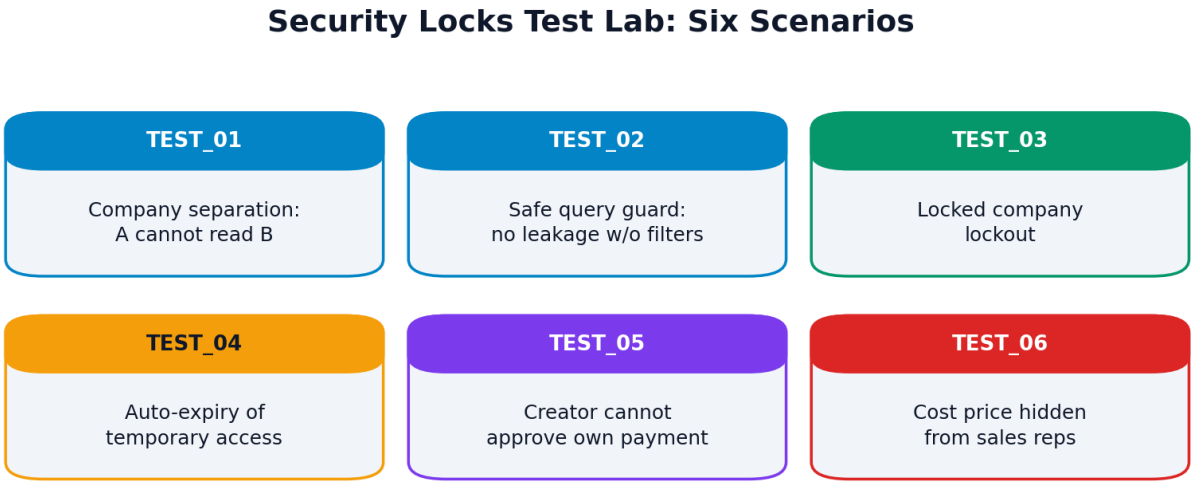


Figure 11 - The six automated scenarios (TEST_01 to TEST_06).

7. Data Model

Domain types live in `src/types/`. The table summarises the main interfaces and their notable fields.

Domain	Key types	Notable details
Entity	BusinessEntity, BankAccount, EntitySummaryKPI	type: manufacturing, retail, services or holding; status: active, archived or sandbox; per-entity currency and fiscal year start.
Auth	UserPersona, EntityAccessGrant, BreakGlassSession, AccessRequest, PlatformRole, PermissionKey	11 roles, 30 permission keys; grants carry status, expiry, justification and scope flags.
Finance	ChartAccount, JournalEntry, JournalLine, ARInvoice, APBill, FiscalPeriodStatus	Account categories ASSET, LIABILITY, EQUITY, REVENUE, EXPENSE; intercompany flags on journals, invoices and bills.
Inventory	InventoryItem, Warehouse, StockAdjustment, IntercompanyStockTransfer	FIFO or weighted-average valuation; stock by warehouse; transfer status in_transit, received or rejected.
Sales / Procurement	Customer, Vendor, PurchaseOrder, SalesOrder	Credit limit and days; vendor rating 1-5; PO status chain and three-way-match status.
HR	Employee, LeaveRequest, PayrollRun	Leave types annual, sick, unpaid, emergency; payroll status draft, processed or disbursed.
Inter-company	IntercompanyTransaction, IntercompanyPairBalance, EliminationEntry	Types TRADE, LOAN, MANAGEMENT_FEE, STOCK_TRANSFER; FX rate to USD.
Approvals	ApprovalRule, ApprovalItem	Document types, threshold, required role, SoD flag, escalation hours, SLA deadline.
Audit	AuditLogEntry, HashVerificationResult	Actions include CREATE, UPDATE, DELETE, APPROVE, REJECT, BREAK_GLASS, ACCESS_GRANT, INTERCOMPANY_POST.
Templates	IndustryTemplate, BusinessOnboardingInput	Template modules, tags, sample roles and recommended chart size.

8. Inter-Company Engine

When one entity trades with another, accountants normally have to enter the transaction twice and later reconcile and eliminate it in consolidation. NEXORA models this as a single action: `postIntercompanyTrade(source, target, amount, memo)` creates an inter-company transaction numbered `IC-xxxxxx`, links the source document to a mirrored document on the target, updates the pair balance between the two entities and records an INTERCOMPANY_POST audit block.

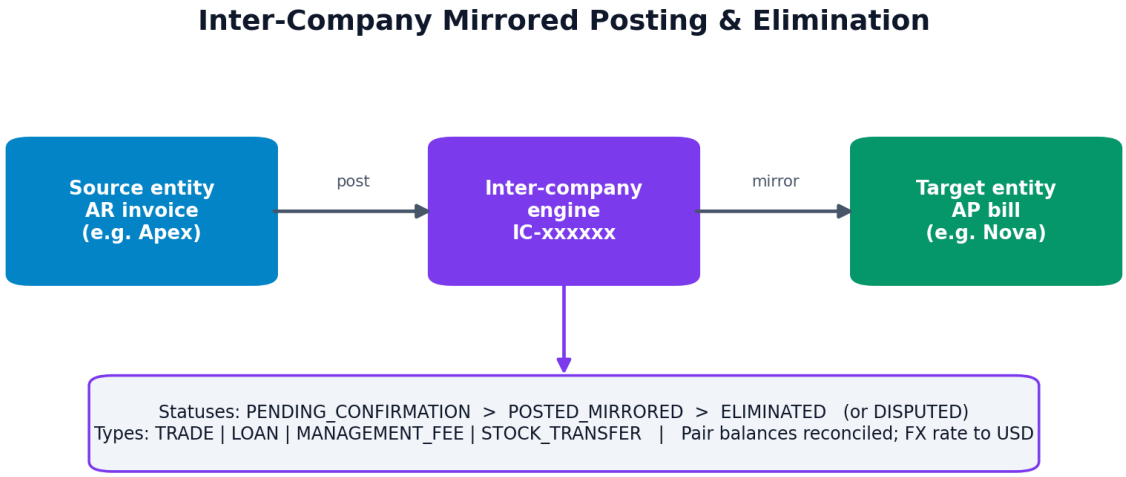


Figure 12 - A trade is posted once, mirrored on the counterparty and later eliminated on consolidation.

The module also shows pair balances (who owes whom, whether reconciled and any unreconciled difference) and elimination entries with debit and credit accounts per period. Transactions can be disputed with a reason. The seed data includes clearing accounts on both sides (an intercompany clearing liability at Apex and an intercompany receivable at Nova) so the demo starts with a realistic position.

9. Nexora Studio (Business Onboarding)

Studio lets an administrator launch a new business entity from one of three industry templates. The onboarding form captures legal name, code, type, currency, country, tax ID, fiscal year start, template, admin email and opening cash balance. Submitting calls `onboardNewBusiness`, which generates an entity ID, creates the entity and writes an audit record. Existing structures can also be cloned.

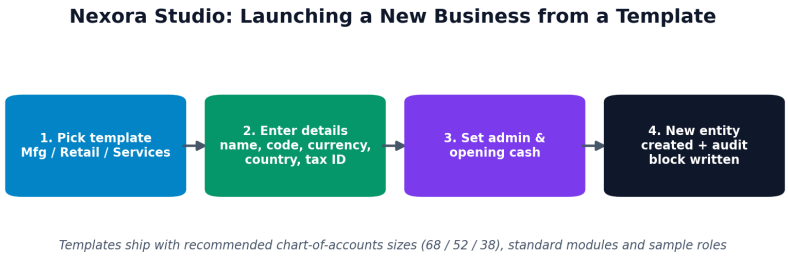


Figure 13 - New-business onboarding flow.

Template	Type / currency	Modules	Chart size
Industrial & Discrete Manufacturing	manufacturing / USD	General Ledger, AP/AR, Inventory & Bins, Procurement & 3-Way Match, Production Floor	68 accounts
Omnichannel Retail & Distribution	retail / EUR	POS Clearing, Warehouse Logistics, Sales & CRM, Procurement, Fast Inventory Valuation	52 accounts
Professional Services & Consulting	services / GBP	Client Invoicing, Time Tracking, Expense Reimbursables, Partner P&L, HR & Payroll	38 accounts

10. Demo Personas and Access Passport

A persona switcher in the header lets a reviewer experience the same data through different identities. Each persona has a deliberately different passport so that the security rules are visible in practice.

Persona	Title / base role	Passport and behaviour
Ahmed Al-Mansoor	Platform Owner and Group Chief Architect (platform_owner)	Business admin on all three entities; MFA enabled; can use consolidated reports, verify the audit vault and execute break-glass.
Sara Lindqvist	Finance Manager (finance_manager)	Approved for Apex; locked for Nova (requires Europe CFO approval) and Zenith. Good for testing access requests and approvals.
Ali Reza Khan	Senior Sales Executive (sales_executive)	Approved only for Nova, with cost price and salary hidden; MFA disabled in the seed, illustrating a weaker account.
Fatima El-Amrani	Head of People and HR Operations (hr_manager)	Approved only for Zenith; can view and process payroll there and sees salary values.
Khalid Benjelloun	Senior Group Compliance Auditor (group_auditor)	Approved on all three entities for 30 days (15 Sep to 15 Oct 2026) with a justification; demonstrates time-boxed access and automatic expiry.

Note: Khalid's seeded access window ends on 15 October 2026, so after that date the demo will show his grants as expired unless the demo data is reset.

11. Running, Building and Verifying

Requirements: Node.js 20.19 or newer (or 22.12 or newer) and npm.

Commands (from the project README and package.json)

Command	Purpose
npm install	Install dependencies.
npm run dev	Start Vite on port 3000, listening on all network interfaces.
npm run typecheck	Run the TypeScript compiler without emitting files (also aliased as npm run lint).
npm run build	Create a production build in dist/.
npm run preview	Serve the production build locally.
npm run clean	Remove the dist and build directories.

Suggested walkthrough for a first review

- Open the app and watch the loading screen, then read the quick-start banner.
- As Ahmed, open the Command Center and compare entities; switch the scope to a single company.
- Switch to Ali and try to open Apex: the passport blocks him. Look at inventory and note the hidden cost price.
- Switch to Sara, request access to Nova, then approve it as Ahmed in the approval queue and check the audit log.
- Post an inter-company trade from Apex to Nova and inspect the mirrored record and pair balance.

- Verify the audit chain, inject a tamper and verify again; then run the Security Locks Test Lab and launch a new business in Studio.

12. Limitations and Production Roadmap

The project's own README is clear about scope, and this review agrees. The points below separate what the prototype demonstrates from what a production system would need.

Area	Prototype today	Needed for production
Authentication	Persona switcher; no real login or MFA enforcement.	Identity provider, SSO, enforced MFA, session management.
Authorization	Checked in browser code, so a user with dev tools could bypass it.	Server-side enforcement on every API call, ideally row-level security per tenant.
Data storage	Seed data and localStorage per browser.	Relational database with transactions, backups and tenant partitioning.
Audit integrity	Hash chain computed and verified client-side; the log can be rewritten by whoever controls the browser.	Server-side append-only store, external anchoring of chain heads, restricted write access.
Accounting rigor	Simplified posting, FX and elimination logic for demonstration.	Full double-entry validation, period close, FX revaluation, statutory reporting.
Testing	Built-in test lab covers six scenarios; typecheck script available.	Automated unit, integration and end-to-end tests in CI.
Data protection	Sample data only.	Encryption at rest and in transit, privacy controls for employee and financial data.

Possible next steps

- Introduce a backend API (for example Node or a managed backend-as-a-service) and move the context workflows into server services.
- Replace the demo hash chain with a server-side ledger and periodic external anchoring.
- Add reporting: trial balance, consolidated P&L, balance sheet and cash-flow statements with FX translation.
- Expand HR (payroll rules, leave accrual) and inventory (costing layers, serial and batch tracking).
- Add automated tests and a CI pipeline, and split the very large HelpModule and AppContext into smaller units.

Conclusion

NEXORA is a well-structured and unusually security-minded ERP prototype. Its strongest ideas are the entity-scoped access passport, the explicit three-dimensional authorization check, enforced segregation of duties and the verifiable audit chain, all demonstrated against a coherent three-company dataset. As a design reference and demo it is effective; to become a working product it needs a real backend and server-enforced security, which the architecture already anticipates through its clear separation of types, data, libraries and workflows.